

```
$ node breakfast.js --mode=async
```

JavaScript □□□

```
// 00000000 Promise 0 async/await
```

sync

□□□□

promise

□□□□□

await

□□□□□

event loop

□□□□

\$ cat menu.md



01 /



vs □□□□□

02 / Promise

□□□□□then/catch□□□□□

03 / async & await



vs □□

04 / Event Loop

VIP □□□□□□□□□

🚧🚧🚧🚧🚧🚧 async □□□□□□□□□□□□

PART 01



□□□□□□□□□□

□□□□□□□□□

```
1  function makeCoffee() {  
2      console.log("☕☕☕☕...");  
3      const start = Date.now();  
4      while (Date.now() - start < 2000) {} // ☐☐☐  
5      console.log("☕☕☕☕");  
6  }  
7  
8  function prepareBreakfastSync() {  
9      makeCoffee(); // 2s  
10     makeToast(); // 1.5s  
11     fryEgg();     // 1s  
12 }
```

CONSOLE — 4.5s

☕☕☕☕...

☐☐☐☐☐

☕☕☕☕...

☐☐☐☐☐☐

☐☐☐☐☐☐☐☐☐☐☐☐

```

CONSOLE - ~2s

0000...
0000...
0000...

0000000000000000
00000000000000

```

```
0000...  
0000...  
0000...  
  
000000000000  
  
00000000
```

```

0000...
0000...
0000000000000000
0000000000000000

```

Diagram illustrating a 3D array structure with dimensions 3x4x3. The array is represented by a grid of small cubes. The first two rows of the first column are highlighted in green, and the first two rows of the second column are highlighted in yellow.

□□□□□□□□□□□□
□□□□□□□□□□

0000000000



□ □ □ □ □ □ □ □ □ □

▶ 0000000000000000

▶ 000000000000

PART 02

Promise

□□□□□□□□□□

□□□□□□□□

pending

□□□□□□

fulfilled

□□□□resolve("🍌🍌")

rejected

□□□□reject("🍌🍌")

```
1 let myPromise = new Promise((resolve, reject) => {
2   let success = true;
3   setTimeout(() => {
4     if (success) resolve("🍌🍌");
5     else reject("🍌🍌");
6   }, 2000);
7 });
8
9 myPromise.then(result => console.log(result))
10 .catch(error => console.log(error));
```

□□□□□Promise.race + AbortController

```
1  const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
2
3  const fetchWithTimeout = (url, { timeout = 3000 } = {}) => {
4    const controller = new AbortController();
5    const timer = delay(timeout).then(() => controller.abort());
6
7    const request = fetch(url, { signal: controller.signal });
8
9    return Promise.race([request, timer]).catch((error) => {
10      if (error.name === "AbortError") return Promise.reject("□□□□");
11      return Promise.reject(error.message || "□□□□");
12    });
13  };
```

Promise.race

□□□□□□□□□□ — □□□□□□□□□□

AbortController

□□□□□□□□□□□□

PART 03

async / await

□□□□□□□□□□

then **NOGRAPH** → **await**

```
await then async/await then
```



```
1  async function serial() {  
2    const coffee = await makeCoffee();  
3    const toast = await makeToast();  
4    return [coffee, toast];  
5  }
```

TOTAL TIME

$\approx 2 + 1.5 = 3.5s$

□□

token API — □□□□□



```
1  async function parallel() {  
2    const [coffee, toast] = await Promise.all([  
3      makeCoffee(),  
4      makeToast(),  
5    ]);  
6    return [coffee, toast];  
7  }
```

TOTAL TIME

$\approx \max(2, 1.5) = 2s$

□□

token API

Promise.all vs allSettled

	Promise.all	Promise.allSettled
□□□□□	 reject □□□□	□□□□□□□
□□	□□□□	{status, value/reason} □□
□□	□□□□□□□	□□□□□□□□□□

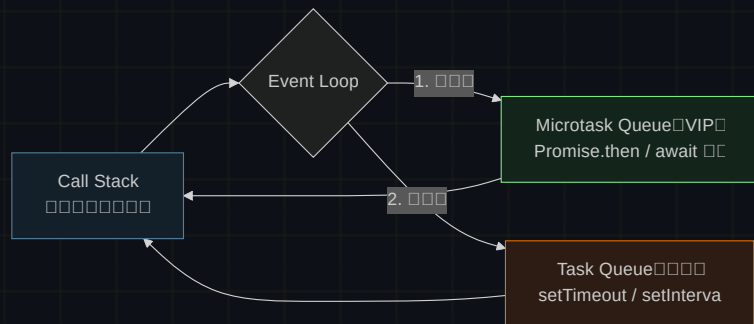
\$ npm run all □□□□□□□□allSettled □□□□□□□

PART 04

Event Loop

0000 VIP 0000000000

□ □ □ □ □ □ □ □ □



□ □ □ □ **VIP** □
□ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □
□ □ □ □ □ □ □ □ □ □ □



```
1 console.log("1");
2 setTimeout(() => console.log("2"), 0);
3 Promise.resolve().then(() => console.log("3"));
4 console.log("4");
```

ANSWER

1 → 4
→ 3
→ 2

1 → 4 → 3 → 2

[illegible]

□□□□□pending → fulfilled / rejected

await Promise.all

□ □ □ □ □ □ □ □ □ □ □ □



Breakfast served.

fetch AbortController

```
$ node breakfast.js --mode=async
```

promise → await → race → event loop